Learning to Play Othello: Rule-Based Bots, Classical Reinforcement Learning and Beyond

Palak Raisinghani

October 2, 2026

Abstract

Othello has simple rules, but its strategy is deep and often counter-intuitive. Moves that bring high immediate rewards can prove to be harmful later, and moves that seem to be of no great immediate value can prove extremely beneficial later on in the game. Othello is thus a strong benchmark to prove capabilty in learning delayed rewards. This project tests several different Rule Based and RL models on their ability to learn how to play Othello. Finally we develop a solid model for the game and offer an attractive interface to play the game against our RL-build Model that we call *Oroletto*.

Contents

1 The Game of Othello **2**
- 1.1 Board and pieces . . . 2
- 1.2 Rules . . . 2
- 1.3 Strategic concepts . . . 3
- 1.4 Board size and solved results . . . 3

2 Why Reinforcement Learning for Games **3**
- 2.1 Games as reinforcement-learning problems . . . 3
- 2.2 Landmark game-playing systems . . . 4
- 2.3 Why Othello . . . 5

3 Rule-Based Bots **5**
- 3.1 The random bot . . . 5
- 3.2 The heuristic bot . . . 5
  - 3.2.1 Evaluation function . . . 5
  - 3.2.2 Tuning the weights . . . 6
- 3.3 The minimax bot . . . 6
  - 3.3.1 Choosing the search depth . . . 7
- 3.4 Head-to-head comparison . . . 8

4 Traditional Training Methods **10**
- 4.1 Common setup . . . 10
- 4.2 The methods . . . 10
- 4.3 Results . . . 11
  - 4.3.1 Comparison against the three rule-based bots . . . 11

4.3.2 Learning dynamics . . . 13
4.3.3 Why the tabular methods fail . . . 13
4.3.4 Why linear function approximation plateaus . . . 14
4.3.5 Why DQN is the strongest, and still loses to search . . . 15
4.3.6 The disc-count ablation . . . 15

5 System Architecture **16**
5.1 Environment design . . . 16
5.2 Evaluation protocol . . . 17
5.3 Correctness validation . . . 17
5.4 Simulator throughput . . . 18
5.5 Playing in the terminal . . . 18

6 Next Steps **18**

1 The Game of Othello

1.1 Board and pieces

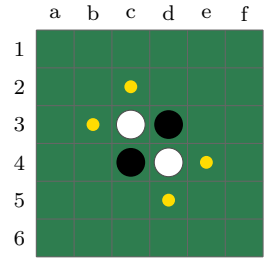
Othello is a two-player, perfect-information game played on a square board, classically 8×8. The pieces are discs that are black on one side and white on the other. The game starts with four discs on the four centre squares, placed diagonally. Black moves first. We name squares by column letter and row number, so on 6×6 the corners are a1, f1, a6 and f6 (Fig. 1a). The rules do not depend on the board size, and this project uses 6×6, 8×8 and 10×10.

1.2 Rules

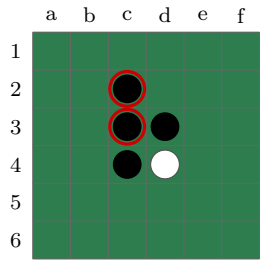
1. **Placing and capturing.** On your turn you place one disc of your colour on an empty square. The placement is legal only if it *sandwiches* at least one straight, unbroken line of opponent discs between the new disc and another of your discs. Lines run horizontally, vertically or diagonally. Every sandwiched disc, in every direction at once, is flipped to your colour (Fig. 1b).

2. **Passing.** A player with no legal placement must pass. Passing is not allowed when a legal move exists.

3. **End of game.** The game ends when neither player can move, usually because the board is full. The player with more discs wins, and equal counts are a draw.



(a) Start position. Dots: Black's legal moves.



(b) After Black plays c2. The white disc on c3 is sandwiched and flips.

Figure 1: The rules on a 6×6 board. This is the same example as the interactive tutorial (`othello --tutorial`, §5.5).

1.3 Strategic concepts

Four ideas recur throughout this report. Each is defined here once.

Stability and corners.
A disc is *stable* if it can never be flipped again. A corner disc is always stable, because no line can sandwich it. A stable corner also lets the discs along its edges become stable.

X- and C-squares.
The squares diagonally adjacent (X) and orthogonally adjacent (C) to a corner. Playing there while the corner is empty tends to give the corner to the opponent.

Mobility.
The number of legal moves a player has. Keeping your own mobility high and the opponent's low eventually forces the opponent onto X- and C-squares.

The disc-count trap.
Owning many discs in the middle game is usually a *weakness*. It flips discs the opponent can flip back, and it gives the opponent more moves. Only the final count matters, so maximising discs early is a classic beginner's error. This error turns out to matter for learning agents as well.

1.4 Board size and solved results

8×8 Othello has roughly 10^{28} legal positions and a game tree of about 10^{58} [1]. It was only recently weakly solved, as a draw under perfect play [18]. The 6×6 game was solved much earlier: it is a win for the second player (White), 16–20 [6]. We start on 6×6 because games are short and simulation is cheap. The board is still far too large to enumerate, so it remains a real learning problem.

2 Why Reinforcement Learning for Games

2.1 Games as reinforcement-learning problems

Reinforcement learning studies agents that learn to act by trial and error, in order to maximise a cumulative reward [17]. A board game fits this framework exactly. The *state* is the position, the *actions* are the legal moves, the *reward* is $+1/0/-1$ for a win, draw or loss at the end of the game, and the opponent is part of the environment. Board games offer two things that real-world RL problems rarely do:

- **A perfect, free simulator.** Rules are exact and games can be replayed millions of times, so data is limited only by compute.
- **Self-play.** An agent can play against copies of itself. It then always faces an opponent of its own strength, which produces an automatic curriculum without human data.

Games are also hard in a way that makes them useful benchmarks. The reward is *sparse* and *delayed*: a single number after dozens of moves. The learner must therefore solve the *credit-assignment problem*, which is working out which earlier moves caused the result. State spaces are also far too large to tabulate, so the agent has to *generalise* from positions it has seen to positions it has not.

2.2 Landmark game-playing systems

Table 1: Landmark game-playing systems. Not all of them use RL; Deep Blue is included as the classic *search-only* contrast.

Game	System	Year	Core idea
Checkers	Samuel's player [12]	1959	Learned a linear evaluation by comparing it with deeper search; an early ancestor of temporal-difference learning.
Othello	IAGO [10]	1982	Hand-built evaluation that introduced mobility as a central concept.
Backgammon	TD-Gammon [19]	1995	A neural network trained by TD(λ) through self-play; reached top human level.
Othello	Logistello [3, 4]	1997	Evaluation fitted by regression on pattern features from self-play games; beat the world champion 6–0.
Chess	Deep Blue [5]	1997	Massive alpha-beta search with a hand-tuned evaluation; no learning. Beat Kasparov.
Atari	DQN [9]	2015	Deep Q-learning from pixels, using experience replay and a target network.
Go	AlphaGo [14]	2016	Policy and value networks (supervised, then RL) guiding Monte Carlo tree search.
Go	AlphaGo Zero [15]	2017	Pure self-play from random play, with a single residual network plus MCTS.
Chess, shogi, Go	AlphaZero [16]	2018	The same algorithm and settings master all three games.
Dota 2	OpenAI Five [2]	2019	Large-scale PPO self-play in a team game with hidden information.
StarCraft II	AlphaStar [20]	2019	Multi-agent RL with a league of past and exploiter agents.
Atari, board games	MuZero [13]	2020	AlphaZero-style planning with a *learned* model of the rules.
Othello	Othello-GPT [8]	2023	A sequence model trained on move lists forms an internal board representation.

Table 1 shows a clear trajectory. Early systems learned only the *weights* of hand-designed features (Samuel, Logistello). TD-Gammon showed that a neural network could learn its own features from self-play. The AlphaGo line then combined learned networks with *search*: the network supplies intuition about which moves are promising and how good a position is, and tree search corrects the network's mistakes by looking ahead. Deep Blue is a reminder that search alone, with expert-tuned evaluation, is already very strong. This is why our ladder of rule-based bots (§3) includes a search bot.

2.3 Why Othello

Othello sits at a useful point between toy problems and grand challenges. The rules fit in three lines, but the state space is far beyond a lookup table even on 6×6. Its strategy is also counter-intuitive: the disc-count trap (§1.3) means the obvious objective is the wrong one. This tests whether a learner discovers the real structure of the game or only the surface statistic. Finally, its small boards are solved, which gives ground truth that large games like Go do not have.

3 Rule-Based Bots

Every learning method is judged against fixed, rule-based opponents, so their strength must be known and stable. We built three, of increasing strength: a random bot, a one-ply heuristic bot and an alpha-beta minimax bot. All results in this section use the shared evaluation protocol summarised below and detailed in §5.2.

Evaluation protocol (summary). Every match is played from K openings of six random plies, and each opening is played once with each colour. We report the *score* (win = 1, draw = $\frac{1}{2}$) with a 95% confidence interval, the win rate and the mean final disc margin.

3.1 The random bot

The random bot plays a uniformly random legal move. It is the floor of the ladder: any agent that has learned anything should beat it clearly. It also calibrates the game itself. Under random play the first mover scores slightly below half on every board size (Table 12), so evaluations must alternate colours.

3.2 The heuristic bot

The heuristic bot is *greedy*. For each legal move it evaluates the resulting position with a hand-written evaluation function and plays the best-scoring move. It looks exactly one ply ahead. The same evaluation, with different weights, also gives the "disc-count-greedy" bot used during tuning (all weight on discs), which is the *easy* level of the terminal game.

3.2.1 Evaluation function

The evaluation is a weighted sum of four features. Each is computed as "own minus opponent" and is defined for any board size:

- **Disc count**: material on the board.
- **Mobility**: the number of legal moves available. High mobility means many options. Low opponent mobility forces the opponent into bad squares.
- **Corners**: corner discs can never be flipped. They are *stable* and anchor the stable discs along their edges.
- **X-squares**: discs diagonally adjacent to an *empty* corner. Such a disc typically hands that corner to the opponent, so its weight is non-positive.

The heuristic bot breaks ties at random and plays a uniformly random move with probability ε=0.05. It is therefore not one deterministic line that a tabular learner could memorise and exploit. Exploitability of that kind would give the learner a misleadingly high win rate.

3.2.2 Tuning the weights

Table 2: Grid search over heuristic weights on 6×6 (96 candidates, 400 games per opponent, ±95% CI ≈ 4–5 points per cell). Scores (%) against a disc-count-greedy bot, the untuned hand default and the random bot. Middle block: the selected weights with one feature removed. Bold: best shown per column.

	Weights				Score vs. (%)			
Configuration	disc	mob.	corner	X-sq.	disc-greedy	hand def.	random	mean
Rank 1 (selected)	1	5	50	-25	85.5	57.1	90.5	**77.7**
Rank 2	0	2	50	-25	85.1	56.6	90.6	77.5
Rank 3	1	5	50	-10	**87.1**	56.2	88.9	77.4
Rank 4	0	2	50	-10	85.6	55.2	90.2	77.0
Rank 5	1	2	50	-10	84.5	**58.9**	87.4	76.9
Selected, no disc	0	5	50	-25	83.6	57.4	88.8	76.6
Selected, no mobility	1	0	50	-25	82.2	47.0	88.2	72.5
Selected, no corner	1	5	0	-25	55.0	30.1	62.9	49.3
Selected, no X-square	1	5	50	0	84.4	50.4	**91.6**	75.5
Disc count only	1	0	0	0	48.5	16.6	59.0	41.4
Hand default	1	5	25	0	84.1	51.1	90.2	75.2

Table 2 gives a clear ranking of feature importance. Corner occupancy dominates. Removing it hurts far more than removing any other feature and leaves a bot close to the disc-only player. Corners matter so much because of *stability*: a corner disc is permanent, and it lets the adjacent edge discs become permanent too. An evaluation that ignores corners optimises quantities that the opponent can reverse in one move.

Mobility is the second most important feature. Removing it hurts most against the hand-default bot, the opponent that itself values corners and mobility. We attribute this to mobility's *tempo* role. Restricting the opponent's options eventually forces them onto X- and C-squares that give up corners. This effect matters most against an opponent that already guards the corners, while weak opponents give corners away even without that pressure.

The disc-count weight barely matters, and the disc-only bot is the weakest configuration in the table, barely better than random. This is the classical *disc-count trap*. Grabbing material mid-game flips many discs that the opponent can flip back, and it increases the opponent's mobility. This result motivates the disc-count ablation for linear function approximation in §4.

The X-square penalty gives a smaller but consistent gain against the hand-default bot. That fits its role of refining corner play, since it only matters when the opponent is able to take corners.

Caveat. The top five configurations differ by less than the per-cell confidence interval. The ranking within that group is therefore noise-level. Selecting the best of 96 noisy candidates also inflates the selected configuration's score (a winner's-curse bias). The selected weights are a sound reference opponent but not a provably optimal one.

3.3 The minimax bot

The minimax bot adds *lookahead*. It searches the game tree d plies deep, assuming both sides play their best, and scores the leaves with the tuned heuristic evaluation (or the exact result at the end

of the game). Alpha-beta pruning [7] skips branches that cannot change the decision. Ordering moves statically (corners first, X- and C-squares last) makes the pruning more effective. Ties at the root are broken at random.

3.3.1 Choosing the search depth

Table 3: Alpha-beta minimax at depth d against the heuristic bot on 6×6 (200 paired openings = 400 games). Nodes per move and time per game were measured on 10 separate games. EBF: the ratio of nodes at depth d to nodes at $d-1$ (effective branching factor). Bold: best score.

d	Score (%)	W/D/L	Margin	Nodes/move	EBF	ms/game
1	56.4 ± 4.9	217/17/166	+2.3	4	–	3
2	73.5 ± 4.3	291/6/103	+9.9	16	3.7	7
3	86.1 ± 3.4	344/1/55	+17.2	68	4.3	23
4	88.2 ± 3.2	352/2/46	+17.3	166	2.4	56
5	**91.2 ± 2.8**	363/4/33	+20.5	614	3.7	210

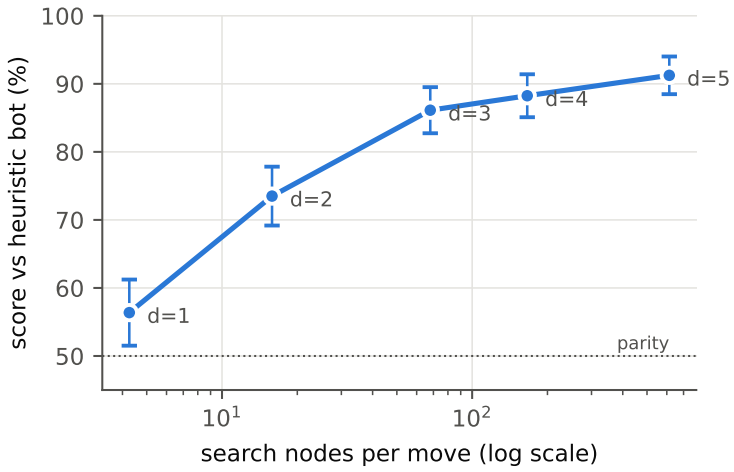


Figure 2: Strength vs. search cost for minimax depths 1–5 on 6×6. Error bars: 95% CI.

Table 3 and Fig. 2 show strength rising monotonically with depth, with clearly diminishing returns per node. The first two extra plies give most of the gain, and later plies each cost several times more search for a smaller improvement. Depth 1 is a useful internal check. A one-ply search with the same evaluation *is* the greedy heuristic bot. It scores only slightly above parity, and we attribute the residual edge to the opponent's ε-random moves.

We attribute the early gains to the *horizon effect*: an evaluation is blind to consequences that fall just beyond its search depth. A greedy bot cannot see that a move gives the opponent a corner on the very next turn, and a two- or three-ply search can. Once that blunder class is covered, extra depth only refines decisions that the evaluation already ranks reasonably, and the gains flatten.

The effective branching factor alternates between larger and smaller values from one depth to the next rather than staying constant. We hypothesise this is alpha-beta's *odd-even effect*: pruning

efficiency differs depending on whether the last ply belongs to the searching side or its opponent. We have not verified this with a separate experiment.

We keep **depth 4** as the reference minimax. It sits after the steep part of the curve, and depth 5 costs several times more per game. This matters because minimax will be evaluated thousands of times during RL training.

3.4 Head-to-head comparison

The opponents must form a clear strength ladder before any RL result can be interpreted. We originally required that heuristic beat random in at least 95% of games and that minimax-d4 beat heuristic in at least 80%. The full round robin was played on all three board sizes with the weights tuned on 6×6.

The minimax criterion passed easily at every size. The heuristic criterion *failed at every size*. On 6×6 the 95% confidence interval lies wholly below the bar, so this was a real shortfall, not sampling noise. The two diagnostics below look for the cause. The resulting, revised gate is reported in Table 6.

Table 4: Diagnostic: win rate (%) and mean disc margin against the random bot for variants of the heuristic bot (6-ply random openings).

	6×6 (1000 games)		8×8 (400 games)	
Agent	Win	Margin	Win	Margin
heuristic ε=0.05	89.6	+17.5	91.5	+29.3
heuristic ε=0	90.8	+18.6	94.8	+31.3
minimax-d1	89.1	+17.5	95.2	+31.9
minimax-d2	**94.2**	+21.1	**99.8**	+38.9

Table 5: Diagnostic: heuristic vs. random on 6×6 as a function of the number of random opening plies (1000 games per row). "Distinct" counts the unique openings among the 500 generated.

		$\varepsilon = 0.05$		$\varepsilon = 0$	
Opening plies	Distinct	Win (%)	Score (%)	Win (%)	Score (%)
0	1	93.1	93.8 ± 1.5	94.1	94.5 ± 1.4
2	12	93.0	93.8 ± 1.5	92.7	93.5 ± 1.5
4	211	89.6	90.6 ± 1.8	91.1	92.0 ± 1.7
6	473	89.6	90.5 ± 1.8	90.8	91.6 ± 1.7

We tested three candidate causes.

1. **Exploration noise.** Removing the ε-random moves helps only slightly (Tables 4 and 5), so it is not the main cause. As expected, the greedy bot and minimax-d1 perform the same, a second consistency check on the search code.

2. **Opening luck.** A 6×6 game lasts only about two dozen plies (Table 12), so six random opening plies are a substantial fraction of it. Table 5 supports this *partially*. Fewer random plies raise the win rate, but only modestly, and the rate stays below 95% even from the plain start position. Short openings also collapse diversity: two plies give only about a dozen distinct openings.

3. **Myopia (horizon effect).** This is the remaining and, we infer, dominant cause. A one-ply evaluation cannot see the opponent's reply, so even a random opponent is sometimes handed a corner. Lookahead is the only change that substantially closes the gap: depth 2 nearly reaches the bar on 6×6 and clears it comfortably on 8×8 (Table 4).

6×6 has the largest gap of the three sizes. We attribute this to the short game, which limits the number of decisions in which the better player can pull ahead.

Resolution. A greedy one-ply bot cannot reliably reach 95% against random on 6×6. The 95% bar was a prior guess, not a property of the game. We therefore revised the heuristic criterion to a **score of at least 90%** (draws count half). For consistency the minimax criterion is also stated on score. We kept the 6-ply openings because evaluation diversity matters more to the later studies than a few points of heuristic strength. We rejected replacing the heuristic bot with minimax-d2. A weaker-but-solid training opponent leaves more headroom for the learning curves in §4, and minimax-d4 already provides the strong reference.

This threshold was chosen after seeing the data. On 6×6 the heuristic bot clears the revised bar on score only narrowly. Its raw win rate is just below 90%, because draws are frequent on the small board. Read the heuristic–random gap as "clear and stable", not "overwhelming".

Table 6: Round robin between the reference opponents. Wins/draws/losses, win rate and score are from the first-named agent's perspective. Games = 2× openings. Gate: the revised threshold on *score* and its outcome.

Board	Match-up	Games	W/D/L	Win (%)	Score (%)	Margin	Gate
6×6	minimax-d4 vs heuristic	1000	870/10/120	87.0	87.5 ± 2.0	+17.2	≥80 PASS
	minimax-d4 vs random	1000	988/3/9	98.8	99.0 ± 0.6	+24.1	–
	heuristic vs random	1000	896/18/86	89.6	90.5 ± 1.8	+17.5	≥90 PASS
8×8	minimax-d4 vs heuristic	400	396/0/4	99.0	99.0 ± 1.0	+41.1	≥80 PASS
	minimax-d4 vs random	400	400/0/0	100.0	100.0 ± 0.0	+43.9	–
	heuristic vs random	400	366/7/27	91.5	92.4 ± 2.6	+29.3	≥90 PASS
10×10	minimax-d4 vs heuristic	200	199/0/1	99.5	99.5 ± 1.0	+61.6	≥80 PASS
	minimax-d4 vs random	200	200/0/0	100.0	100.0 ± 0.0	+68.6	–
	heuristic vs random	200	186/0/14	93.0	93.0 ± 3.5	+41.5	≥90 PASS

Table 7: Bradley–Terry Elo fitted to each size's round robin (random = 0). Near-perfect scores make the larger gaps poorly determined; a weak prior keeps them finite.

Board	random	heuristic	minimax-d4
6×6	0	398	740
8×8	0	433	1217
10×10	0	445	1287

Tables 6 and 7 confirm the intended ordering random < heuristic < minimax on every board size. The gap between minimax and heuristic widens as the board grows. We attribute this to two effects. Larger boards mean longer games with more decisions, so lookahead has more chances to pay off. And the weights were tuned on 6×6, which likely suits the greedy bot less on bigger boards, while search partly makes up for a mis-tuned evaluation.

4 Traditional Training Methods

This section trains five classical RL methods on 6×6. All of them use the same fixed opponent and the same budget, and each is then compared against all three rule-based bots. The question is not only how well each method plays, but *why* it stops improving where it does.

4.1 Common setup

- **Opponent and episodes.** Each learner plays the heuristic bot (§3), alternating colours and starting from random 6-ply openings, for 200,000 training games per run, with 3 seeds per method.
- **The learner's view.** The learner sees only its own decision points. Its state s_t is the position when it is to move, and the opponent's reply is part of the environment's transition to s_{t+1}. The reward is $+1$, 0 or -1 at the end of the game and zero otherwise, with no discounting ($\gamma = 1$).
- **Exploration.** ε-greedy, with ε decaying linearly from 0.3 to 0.05 over the first half of training.
- **Evaluation.** Every 5,000 games the greedy policy plays 200 evaluation games against the heuristic bot (for the learning curves). At the end it plays 1,000 games against random and against heuristic, and 500 against minimax-d4 (for the comparison). The evaluation openings are fixed and shared across methods.

4.2 The methods

Tabular Q-learning [21]. Keeps a table entry $Q(s, a)$ for every state–action pair it has visited and updates it toward a bootstrapped one-step target:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\big[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)\big].$$

The max makes it *off-policy*: it learns the value of the greedy policy while it behaves ε-greedily. To give the table its best chance, each position is stored in a canonical form under the 8 symmetries of the board, so symmetric positions share one entry. Unvisited entries read as 0.

SARSA [11]. Identical to Q-learning except that the target uses the action a_{t+1} the agent *actually* took next, i.e. $r_{t+1} + \gamma Q(s_{t+1}, a_{t+1})$. It is therefore *on-policy*: it values the exploring policy itself, including its exploratory mistakes.

First-visit Monte Carlo control [17]. Does not bootstrap. After the game ends, every visited pair is moved toward the observed return G_t (here the final result) by a sample average, $Q(s, a) \leftarrow Q(s, a) + \frac{1}{N(s,a)}\,[G_t - Q(s, a)]$. Its targets are unbiased but have high *variance*: each one is a single noisy game outcome that depends on every later move by both players. (Positions never repeat within an Othello game, so first-visit and every-visit MC coincide.)

Linear function-approximation Q-learning. Replaces the table with a weighted sum of features, $Q(s, a) = \mathbf{w}^\top \boldsymbol{\phi}(s \oplus a)$, where $s \oplus a$ is the *afterstate*: the position right after the learner's move. The features are disc difference, mobility difference, corner difference, X-square difference, stable-edge-disc difference and a bias, each scaled to roughly unit range. The weights are learned by semi-gradient Q-learning, $\mathbf{w} \leftarrow \mathbf{w} + \alpha\,\delta_t\,\boldsymbol{\phi}$. Unlike a table, it *generalises*: every position with similar features gets a similar value, even if it has never been seen. But it can only represent evaluations that are linear in those features.

Vanilla DQN [9]. Replaces the table with a neural network Q_θ: an MLP with two hidden layers of 256 units over three input planes (own discs, opponent discs, legal moves). It outputs one value per square plus pass. Two components stabilise training: an *experience-replay* buffer (200k transitions, sampled uniformly), which breaks the correlation between consecutive positions, and a *target network* Q_{θ^-}, synced every 2,000 updates, which keeps the bootstrap target from chasing itself. The loss is Huber$\big(Q_\theta(s,a) - [r + \gamma \max_{a' \text{ legal}} Q_{\theta^-}(s',a')]\big)$. Illegal actions are masked both when acting and in the max. Unlike the linear model, the network can learn its own features from raw discs.

Disc-count ablation (linear FA). To test the disc-count trap (§1.3) on a learner, the linear method is trained three more ways: without the disc feature, with *only* the disc feature (plus bias), and with the full feature set but rewarded by the final *disc margin* instead of win/loss.

4.3 Results

4.3.1 Comparison against the three rule-based bots

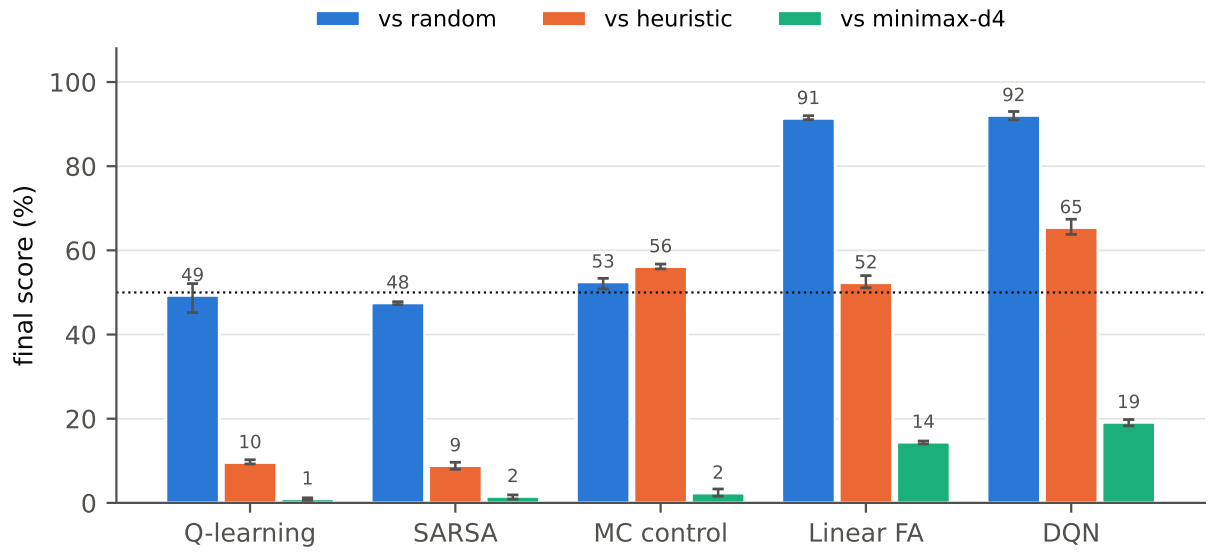


Figure 3: Final score of each traditional method against the three rule-based bots on 6×6. Bars: mean over 3 seeds; whiskers: min–max across seeds; dotted line: 50% (parity). Against random and heuristic each bar is 1,000 games per seed; against minimax-d4 it is 500.

Table 8: Final evaluation of the traditional methods (mean ± std over seeds, %). "Win vs heur." is the raw win rate against the training opponent. "Train" is the wall-clock hours per run on one CPU core. The reference row is the heuristic bot's own result from the rule-based round robin (Table 6). Bold: best per column.

		Score vs. (%)				
Method	Seeds	random	heuristic	minimax-d4	Win vs heur.	Train (h)
Q-learning	3	49.3 ± 3.6	9.6 ± 0.6	1.0 ± 0.3	8.5	0.5
SARSA	3	47.5 ± 0.4	8.9 ± 0.8	1.5 ± 0.6	7.7	0.4
MC control	3	52.5 ± 1.4	56.1 ± 0.6	2.3 ± 0.9	54.8	0.3
Linear FA	3	91.4 ± 0.5	52.3 ± 1.5	14.4 ± 0.4	51.3	0.8
DQN	3	**92.1 ± 1.0**	**65.4 ± 1.8**	**19.1 ± 0.8**	64.0	1.5
Heuristic bot (reference)	–	90.5	–	12.5	–	–

Figure 3 and Table 8 separate the methods into three tiers. The tabular TD methods (Q-learning and SARSA) never leave random play: they draw level with the random bot and lose almost every game to the other two. Linear function approximation reaches the strength of the heuristic bot: it scores about as well as the heuristic does against random, breaks even against the heuristic, and loses to minimax-d4 by a similar margin. DQN is the strongest method on *all three* opponents, and is the only one that clearly beats its training opponent while still dominating random play.

Monte Carlo control does not fit this ordering. It beats the heuristic bot it was trained against, yet against the *random* bot it does no better than a coin flip. A policy that had learned to play Othello would find the random bot the easiest of the three opponents. MC control instead does well only against the one opponent it has seen. We call this *overfitting to the training opponent*: a policy that exploits the specific habits of one opponent instead of learning the game. Evaluating against the training opponent alone would have ranked MC control among the best methods. The three-bot comparison is what exposes it. No method comes close to minimax-d4. Every one of them chooses moves without lookahead, and none overcomes the horizon effect that already limited the one-ply heuristic bot (§3).

4.3.2 Learning dynamics

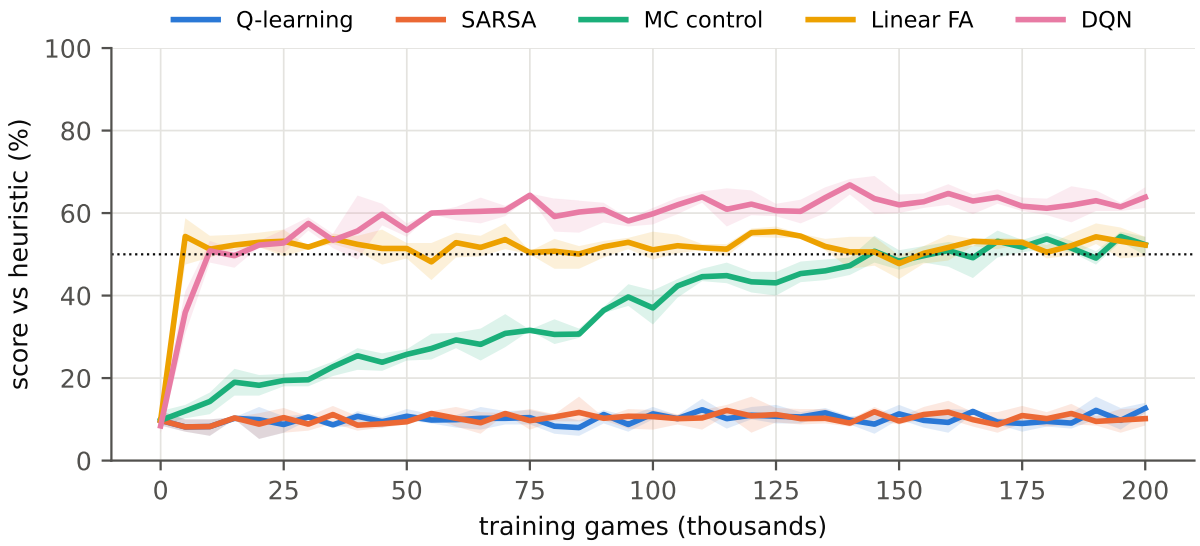


Figure 4: Learning curves: greedy-policy score against the heuristic bot (200 games per point) during training. Lines: mean over 3 seeds; bands: min–max. Q-learning and SARSA overlap almost exactly.

The curves in Fig. 4 show three different failure modes.

- **Flat at the bottom (Q-learning, SARSA).** No measurable improvement over the whole budget.
- **Immediate plateau (linear FA).** Reaches parity within the first evaluation interval and then does not move. Additional data does not help.
- **Slow, steady climb (MC control, DQN).** Both are still rising at the end of the budget, MC control from a much lower start.

4.3.3 Why the tabular methods fail

Table 9: Tabular diagnostics at the end of training (mean over 3 seeds). Coverage is the fraction of the learner's decision positions during the final evaluation games that already have an entry in its table. Update-error variance is the variance of (target $- Q$) over the updates of the first and of the last 5,000 training games.

		Coverage (%)		Update-error var.	
Method	States (10^6)	vs heuristic	vs random	first 5k	last 5k
Q-learning	1.92	36.4	8.2	0.066	0.057
SARSA	1.92	36.6	8.3	0.066	0.058
MC control	1.52	66.9	9.6	0.367	0.622

The curse of dimensionality. Table 9 shows the mechanism directly. After 200,000 games each table holds between one and two million canonical positions, yet in games against the random bot fewer than one in ten of the positions the learner meets are in its table. A lookup table has no notion of similarity between positions, so a position it has not seen is a position it knows nothing about, and it falls back to an arbitrary choice. This is the *curse of dimensionality* in its plainest form: the number of reachable positions is so large that a table cannot be filled from any feasible amount of play. It is the same even after merging the 8 board symmetries. Coverage is markedly higher against the heuristic bot, because a mostly deterministic opponent keeps leading the game back into the same lines. That difference is what MC control exploits.

Why Q-learning and SARSA are indistinguishable. The two update rules differ only in which next-state entry they bootstrap from: the greedy one or the one actually taken. A unit test confirms they produce different targets when the next state's entries exist (`tests/test_rl.py`). In practice the next state is usually unvisited, so both entries read the default value of zero, the two targets coincide and the learners behave the same. We attribute the TD methods' failure to *bootstrapping* under sparse revisits. A one-step target $r + \gamma Q(s', \cdot)$ only carries information backwards one move per visit. The final reward therefore has to travel back through many revisits of the same line before it reaches early positions, and almost no line is revisited that often.

Why MC control behaves differently. MC control does not bootstrap. After every game it moves *all* visited entries straight toward the actual outcome, so a single winning game against the heuristic bot marks every position on that line as good at once. Against a nearly deterministic opponent, starting from a bounded set of openings, the same lines come back often enough for these marks to accumulate. Its coverage against the heuristic bot rises together with its score. It is effectively memorising a growing book of winning lines against *that one bot*. Coverage against the random bot stays low, which is why the memorised book is useless there. The price of not bootstrapping is *variance*. Each MC target is a single, noisy game outcome, and its update-error variance is several times that of the TD methods. That variance is the bias–variance trade-off in action: unbiased targets, but slow and noisy progress, consistent with its slow climb in Fig. 4.

Design note. We added the heuristic bot's ε-randomness and the random openings specifically to stop a tabular learner from memorising lines against it (§3). This was *not sufficient*. MC control still found enough repetition to exploit. Any future result measured only against the training opponent should be treated with suspicion. The curve was still rising at the end of the budget, and we expect, but have not tested, that longer training would increase this exploitation without improving play against other opponents.

4.3.4 Why linear function approximation plateaus

The linear learner *generalises*: two positions with similar features get similar values whether or not either was seen. That is why it beats the random bot as convincingly as the heuristic bot does, after almost no training. But it reaches its ceiling almost immediately and stays there. Its features are essentially the heuristic bot's own features plus edge stability, so the best policy it can represent is a re-weighted one-ply heuristic. Learning finds those weights within a few thousand games, and the learned weights reproduce the ordering found by hand-tuning: corners first, then mobility, with disc count near zero (Table 10). This is a *capacity* or *inductive-bias* limit rather than a data limit. The model class itself contains nothing better than the heuristic, so more games cannot help.

4.3.5 Why DQN is the strongest, and still loses to search

DQN combines the two missing ingredients. It generalises across positions like the linear model, but it learns its own features from raw disc planes instead of being restricted to hand-chosen ones. Replay lets each game be learned from many times, and the target network keeps the bootstrapped targets stable while it does. The result is the only method that improves on the heuristic bot without losing generality. It still chooses each move from a single forward pass, with no lookahead. Minimax-d4 punishes exactly the tactical oversights a one-ply policy cannot see, which is why DQN loses heavily to it. Its curve is still rising, so a longer run would likely help somewhat. However, closing the gap to search is the job of the self-play and MCTS methods in §6, not of more DQN training.

4.3.6 The disc-count ablation

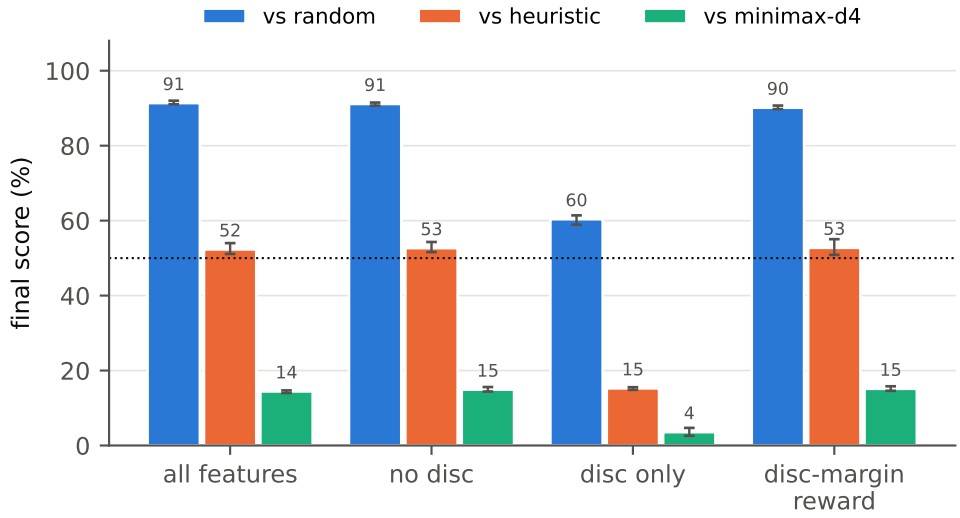


Figure 5: Disc-count ablation for linear function approximation. Same format as Fig. 3.

Table 10: Disc-count ablation (mean ± std over 3 seeds, %), with the learned weights (mean over seeds) on the scaled disc, mobility and corner features.

	Score vs. (%)			Learned weight		
Variant	random	heuristic	minimax-d4	disc	mob.	corner
All features, win/loss reward	91.4 ± 0.5	52.3 ± 1.5	14.4 ± 0.4	-0.05	+0.40	+0.86
No disc feature	91.1 ± 0.4	52.7 ± 1.4	14.9 ± 0.6	–	+0.44	+0.88
Disc feature only	60.4 ± 1.3	15.2 ± 0.4	3.6 ± 1.1	+0.87	–	–
All features, disc-margin reward	90.1 ± 0.5	52.8 ± 2.1	15.1 ± 0.6	+0.00	+0.26	+0.56

Fig. 5 and Table 10 separate two versions of the disc-count trap.

- **Disc-count *features*: the trap is reproduced.** Removing the disc feature changes nothing, and the full model learns to give it essentially zero weight. A learner restricted to disc count *only*

collapses to the strength of the disc-greedy bot from the tuning study, barely above random. Disc count simply does not carry the information needed to play well. The learner rediscovers what the rule-based tuning found (§3).

- **Disc-count *reward*: no measurable harm.** Rewarding the final disc margin instead of the win leaves strength unchanged on all three opponents. This *contradicts* the expectation in our original plan. We attribute it to the features. With corner and mobility features available, the positions that maximise the final margin are the same ones that maximise the chance of winning, so the two objectives agree. The learned weights only shrink in scale, as the smaller reward range would predict, and keep their ordering. The trap lies in valuing discs *during* the game, which the features control, not in scoring them at the *end*, which is all the reward does.

5 System Architecture

All experiments run on one Python code base (package `othello`). It has a size-parameterised environment, the bots, the learners, the evaluation harness, and a terminal front-end for human play.

5.1 Environment design

The environment is shared by every experiment, and each part of it exists for a downstream need:

Bitboard state.
A position is two integers of n^2 bits (one per colour), plus the side to move and a consecutive-pass counter. Move generation shifts the bitboards in the eight compass directions and masks out bits that wrap around a row edge. Nothing depends on a fixed n, so 6×6, 8×8 and 10×10 run the same code. This is the precondition for the size-generalisation study.

Immutable states.
Applying a move returns a new state rather than mutating the old one. Copying a position is therefore free, which tree search (minimax now, MCTS later) relies on to hold many positions at once.

Action space with an explicit pass.
Actions are the n^2 squares plus one *pass* action. Pass is legal only when no placement is. A fixed-size action space with a legality mask lets learned policies mask illegal moves rather than learn to avoid them.

Configurable input planes.
The state can be encoded as any subset of {own discs, opponent discs, empty squares, side to move, legal moves}. The planned input-representation sweep (§6) varies this set without touching the environment.

One agent interface.
Every player implements `act(state)`→`action`. Bots, learned policies and search agents are therefore interchangeable in the evaluation harness.

5.2 Evaluation protocol

All comparisons in this report use the same protocol. A fixed seed generates K opening positions by playing six uniformly random plies from the standard start. Each opening is played twice with colours swapped, which cancels the *first-move advantage*: the systematic edge (positive or negative) that comes from moving first rather than from playing well. Results are reported from the first agent's perspective as wins/draws/losses, *score* (win = 1, draw = $\frac{1}{2}$) with a 95% normal-approximation confidence interval, and mean final disc margin. The random openings keep a deterministic pair of agents from replaying one game $2K$ times. They also stop a learner from memorising a single line against a fixed opponent, which matters for the tabular methods in §4. Six plies give about 95% distinct openings at K=500, against about 42% for four plies. The normal-approximation interval degenerates (to zero width) when a score is exactly 0 or 1. Those cells should be read as "no losses observed", not as certainty.

5.3 Correctness validation

Table 11: Environment correctness checks (48 automated tests, all passing; see `results/baselines/pytest.txt`).

Check	What it guards against
Perft, 8×8, depth 1–6	Errors in move generation or flipping. Leaf counts must equal the published values 4, 12, 56, 244, 1396, 8200.
Naive cross-check, $n \in \{4, 6, 8, 10\}$	Edge-wrapping errors at any board size. On 30 random games per size, the legal-move set and flipped discs must match an independent 2-D grid implementation at every ply.
Forced pass / double pass	Wrong termination. A player with no move must pass; two consecutive passes end the game; a placement resets the pass counter.
Full board, draw	Wrong winner attribution on equal disc counts.
Transpose symmetry	Direction-specific shift bugs. Transposing the board must transpose the legal-move set.
Encoding shapes	Plane-construction errors. Every plane configuration has shape (C, n, n) with the expected disc and legal-move counts.
Agents	Illegal moves from any bot; minimax missing a forced win; the heuristic bot ignoring a corner.

Perft ("performance test") counts the leaf positions of the full game tree to a fixed depth. A single wrong flip or missed move changes the count, which makes it the most sensitive check of the move generator. Perft only exists for 8×8, so the naive cross-check carries the burden for the other sizes. It compares the bitboard engine with an independent implementation that shares no code with it, at every position of real games, including the board edges where wrap-around bugs would appear.

5.4 Simulator throughput

Table 12: Single-core simulator throughput, random vs. random, 1000 paired openings per size. "Black score" is the share of points won by the first mover under uniformly random play.

Board	Games	Games/s	Plies/s	Plies/game	Black score (%)
6×6	2000	**2,142**	56,910	26.6	46.9
8×8	2000	917	49,938	54.4	48.0
10×10	2000	449	40,597	90.4	47.8

Table 12 shows games per second falling steeply with board size while plies per second fall only mildly. Almost all of the slowdown therefore comes from games getting longer: the number of plies grows roughly with the number of squares. Only a small part comes from each move costing more. Cost per ply grows slowly because each bitboard operation acts on all n^2 squares at once and the directional scans take $\mathcal{O}(n)$ steps. This tells us how to budget compute. Simulation will be negligible next to neural-network inference in the self-play studies, and going from 6×6 to 10×10 costs mainly through game length. The pure-Python engine is therefore sufficient, and a compiled engine is not needed yet.

On every board size the first mover scores slightly below half under random play, so the side moving second has a small edge. This is *consistent with* the known result that 6×6 Othello is a second-player win under perfect play. Random play, however, says nothing about perfect play, so we read it only as a reason to keep evaluating with both colours.

5.5 Playing in the terminal

The `othello` command lets a person play any agent in the terminal: `othello --play [1--5 | beginner …expert]` chooses a rule-based bot, `othello --model PATH` loads a trained model (linear or DQN), and `othello --tutorial` teaches the rules with the examples of Fig. 1 before a coached practice game. Options cover the board size (4–10), colour, legal-move hints and a coach mode that flags moves where the expert bot disagrees. The command reuses the same environment and agent classes as the experiments.

6 Next Steps

- **Self-play learning.** Self-play PPO, Double DQN with an opponent pool of past checkpoints, and an AlphaZero-style agent (residual policy–value network, MCTS, symmetry augmentation) on 6×6. We compare them on final strength, sample efficiency, wall-clock time and stability across seeds, and check them against the solved 6×6 result.
- **Architecture and search-budget sweep.** Depth, width, input planes, head design and MCTS simulations, varied one at a time and summarised as a strength-versus-compute Pareto plot.
- **Scaling to 8×8.** Re-tune the rule-based bots for 8×8 (Table 6 shows the 6×6 weights are weaker there) and train the selected configuration. Optionally, play matches against the Edax engine.

- **One network for every board size.** Test zero-shot transfer, then train a fully convolutional, globally pooled network on mixed board sizes and compare it with a small-to-large curriculum. The result is reported as a train-size × test-size win-rate matrix.

References

[1] L. V. Allis. *Searching for Solutions in Games and Artificial Intelligence*. PhD thesis, University of Limburg, 1994.

[2] C. Berner et al. Dota 2 with large scale deep reinforcement learning. arXiv:1912.06680, 2019.

[3] M. Buro. Statistical feature combination for the evaluation of game positions. *Journal of Artificial Intelligence Research*, 3:373–382, 1995.

[4] M. Buro. The Othello match of the year: Takeshi Murakami vs. Logistello. *ICCA Journal*, 20(3):189–193, 1997.

[5] M. Campbell, A. J. Hoane, and F.-h. Hsu. Deep Blue. *Artificial Intelligence*, 134(1–2):57–83, 2002.

[6] J. Feinstein. Amenor wins world 6×6 championships! *British Othello Federation Newsletter*, July 1993.

[7] D. E. Knuth and R. W. Moore. An analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4):293–326, 1975.

[8] K. Li, A. K. Hopkins, D. Bau, F. Viégas, H. Pfister, and M. Wattenberg. Emergent world representations: Exploring a sequence model trained on a synthetic task. In *ICLR*, 2023.

[9] V. Mnih et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.

[10] P. S. Rosenbloom. A world-championship-level Othello program. *Artificial Intelligence*, 19(3):279–320, 1982.

[11] G. A. Rummery and M. Niranjan. On-line Q-learning using connectionist systems. Technical Report CUED/F-INFENG/TR 166, Cambridge University, 1994.

[12] A. L. Samuel. Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, 3(3):210–229, 1959.

[13] J. Schrittwieser et al. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020.

[14] D. Silver et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529:484–489, 2016.

[15] D. Silver et al. Mastering the game of Go without human knowledge. *Nature*, 550:354–359, 2017.

[16] D. Silver et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144, 2018.

[17] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

[18] H. Takizawa. Othello is solved. arXiv:2310.19387, 2023.

[19] G. Tesauro. Temporal difference learning and TD-Gammon. *Communications of the ACM*, 38(3):58–68, 1995.

[20] O. Vinyals et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575:350–354, 2019.

[21] C. J. C. H. Watkins and P. Dayan. Q-learning. *Machine Learning*, 8:279–292, 1992.